MULT20015 Elements of Quantum Computing

Practice Class 7

Welcome to Practice Class 7 of MULT20015 Elements of Quantum Computing, covering exercises relevant to the material presented in lectures in Week 7.

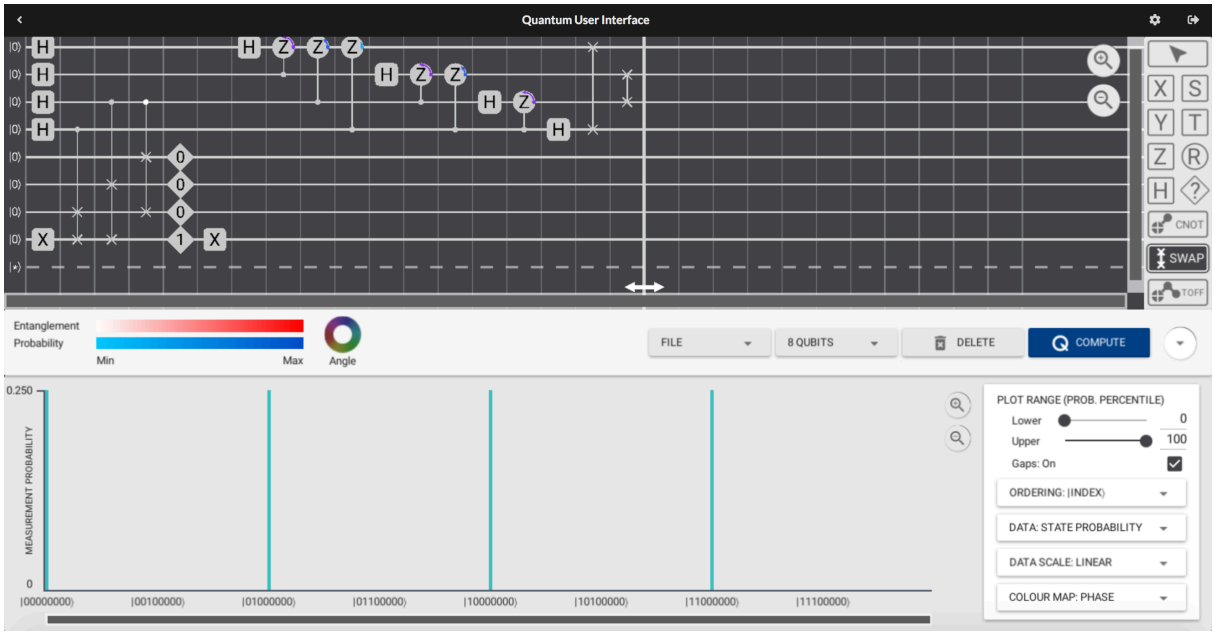
The purpose of this week's exercises is to:

- Implement and investigation a small version of Shor's algorithm
- Implement and investigate the BB84 quantum key distribution protocol.

1 Shor's quantum factoring algorithm

In this exercise we will encode and attempt to understand an instance of Shor's algorithm, albeit a specially designed case to make it not too lengthy on the QUI – we'll call this "Baby Shor". The full circuit for Shor's algorithm including the modular arithmetic is in general quite complicated. However, with some prior knowledge about the number we are factoring we can make some simplifications and still convey the same information. The circuit for Baby Shor is shown in the screenshot and described below (ref: Greg White).

Baby Shor, as one would expect, factors 15 by computing $f(x) = 2^x \bmod 15$ (i.e. the parameter a is set to $a = 2$), measuring out the $|f(x)\rangle$ register (bottom 4 qubits), and then performing a QFT on the resulting superposition over the $|x\rangle$ register (top 4 qubits).



Recall from lectures that the more general circuit for Shor's algorithm has two registers: an $|x\rangle$ register which is 2L qubits, and an L qubit register – initialised to $|1\rangle$, on which we compute $|f(x)\rangle$ (L is the number of bits in N). The first simplification we make is reducing the $|x\rangle$ register to L qubits, a move which will be later justified.

We compute the modular exponential through repeated modular multiplication. Writing $x = x_1x_2x_3x_4 = 2^3x_1 + 2^2x_2 + 2^1x_3 + 2^0x_4$, we do four multiplications on the $|f(x)\rangle$ register, controlled by each qubit of the $|x\rangle$ register. With each step, we multiply $|f(x)\rangle$ by 2^{2^i}, controlled by the x_{4-i} qubit. This is the same as multiplying by $2^{2^i x_{4-i}}$, since if $x_{4-i} = 0$, then we multiply by 1 – which is the same as doing nothing.

Step-by-step, the $|f(x)\rangle$ register follows the pathway:

$$\begin{aligned}|1\rangle \to |1 \times 2^{2^0 x_4} \bmod 15\rangle \to\ & |1 \times 2^{2^0 x_4} \times 2^{2^1 x_3} \bmod 15\rangle \to\ |1 \times 2^{2^0 x_4} \times 2^{2^1 x_3} \times 2^{2^2 x_2} \bmod 15\rangle \\ \to\ & |1 \times 2^{2^0 x_4} \times 2^{2^1 x_3} \times 2^{2^2 x_2} \times 2^{2^3 x_1} \bmod 15\rangle = |2^{2^3 x_1 + 2^2 x_2 + 2^1 x_3 + 2^0 x_4} \bmod 15\rangle \\ =\ & |2^x \bmod 15\rangle\end{aligned}$$

Now comes the main hack : general multiplications are difficult to perform on quantum circuits, but we have chosen a=2 (i.e. $2^x \bmod 15$) to make things easy. Consider each multiplication:

$$\begin{aligned}2^{2^0} \bmod 15 &= 2 \\ 2^{2^1} \bmod 15 &= 4 \\ 2^{2^2} \bmod 15 &= 1 \\ 2^{2^3} \bmod 15 &= 1\end{aligned}$$

We notice firstly that the last two multiplications are multiplications by 1, which is no action. So we need only to compute the first two multiplications. We next notice that the first two are multiplications by powers of 2. There is something special about multiplying binary numbers by a power of 2. If we write $j = 2^3j_1 + 2^2j_2 + 2^1j_3 + 2^0j_4$, then $2^n \times j = 2^{3+n}j_1 + 2^{2+n}j_2 + 2^{1+n}j_3 + 2^{0+n}j_4$. That is, we are shifting each of our numbers to the left by one position. This is not a new concept. Think about how you multiply numbers by 10, or 100 in base 10. This is done by shifting all of your numbers to the left by 1 or 2 positions. Also, recall that binary numbers are inherently defined modulo 2^n.

In terms of gates, shifting our bits is the same as swapping their positions. We start out at 0001 and have a controlled multiplication of 2, which would give 0010. This is the same as swapping x_3 and x_4. Similarly, bit-shifting by two positions means taking $x_1x_2x_3x_4 \to x_3x_4x_1x_2$ – achieved through two SWAPS. Result – our complicated exponentiation circuit can be reduced to just three controlled SWAP gates for this simple case!

This means now that our circuit for Shor's algorithm can be summarised by:
1) a controlled shift by x_4 of 1 position acting on the $|f(x)\rangle$ register
2) a controlled shift by x_3 of 2 positions acting on the $|f(x)\rangle$ register
3) a measurement of the $|f(x)\rangle$ register to isolate one superposition in the $|x\rangle$ register
4) a QFT on the $|x\rangle$ register
5) Measure the $|x\rangle$ register (or in the QUI just inspect the state)

The reason why we can reduce the $|x\rangle$ register down to L qubits is related to the period. A domain of 2^{2L} is usually chosen in order to guarantee that we can find the period

through continued fractions, in the case that we do not have exact periodicity. However, in the simple scenario here the period is 4, which divides 2^4, and so we do not need the extra L qubits.

Finally, note that the X operation on the last qubit is for convenience in reading out the final measurement when the bottom register is measured as 0001, which after the X operation becomes 0000.

Exercise 1.1 Code up Baby Shor in the QUI. Save it and run it several times to understand how it works (you can edit appropriately to verify the function f(x) evaluation), what the output means (you can add measurements on the x register, or just interpret the final state), and how to determine the prime factors of 15.

2 Quantum Communication – the BB84 Protocol in the QUI

An important application of qubits is to enable secure communication between two parties. There are several different quantum communication protocols with varying degrees of complexity. In this section, we will implement a very simple protocol known as "BB84" which is based on single qubits and can be implemented in the QUI. BB84 stands for Bennett and Brassard, the names of the two mathematicians who proposed this protocol in 1984.

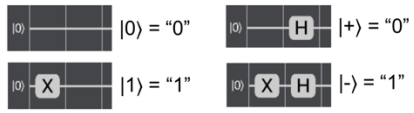
Basic idea: Alice wants to send Bob a secret key (string of bits) over a channel that might be intercepted by an eavesdropper (Eve). Using qubits Alice can encode her bits in randomly selected instances of either the Z basis ($|0\rangle$, $|1\rangle$) or X basis ($|+\rangle$, $|-\rangle$). By the rules of quantum superposition, if Alice prepares a bit in one basis and Bob measures in a different basis he is not guaranteed of measuring the correct bit of information. Only when the bases agree will Bob measure Alice's bit with 100% certainty (see below). The trick is that after measurement Alice and Bob only compare measurement bases, not the actual bits measured. They then use a certain number of correct bits to test whether an eavesdropper tried to intercept and re-send the communication – in this case Alice and Bob will see the percentage of correct bits drop.

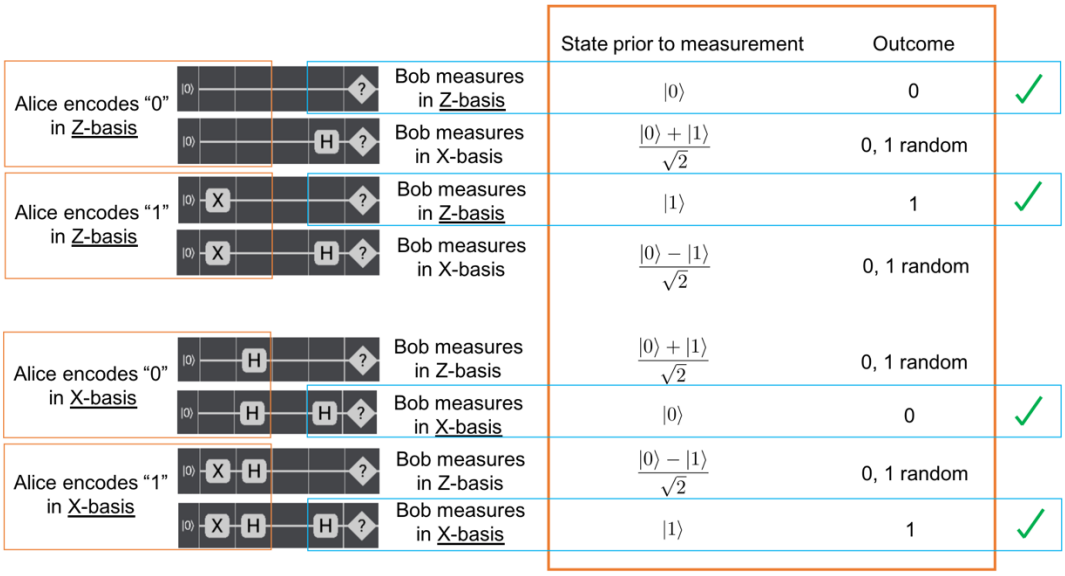
Key points of BB84: Bit coding, basis, and measurement

Alice encodes bits in either the Z or X bases

Bit	Z-basis	X basis
0	$\|0\rangle$	$\|+\rangle = (\|0\rangle + \|1\rangle)/\sqrt{2}$
1	$\|1\rangle$	$\|-\rangle = (\|0\rangle - \|1\rangle)/\sqrt{2}$

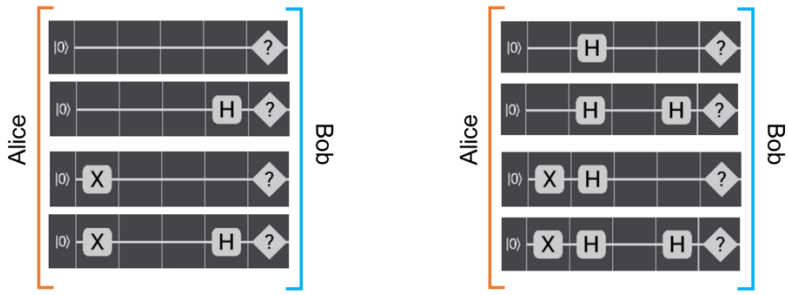
H-gate converts between the Z and X bases (Lecture 2)





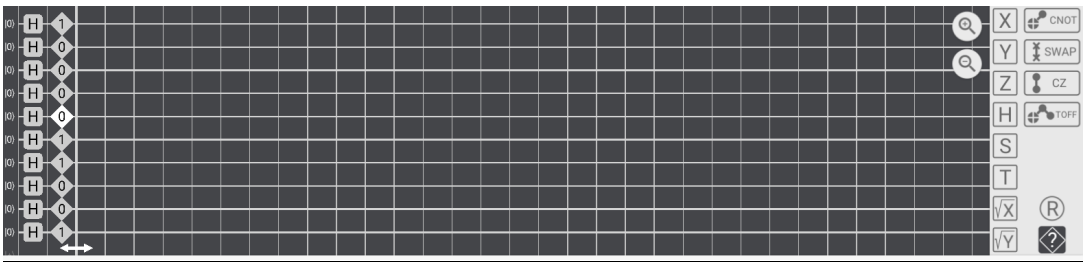
-> Bob obtains the correct bit when he measures in the same basis that Alice uses to encode.

Exercise 2.1 For a single qubit, code the BB84 protocol for the no-eavesdropper case in the QUI and repeat for all eight cases of bases choice by Alice and Bob (illustrated below). In single qubit mode, use the time scrubber to examine the qubit state during the protocol to make sure you understand how it works.



Before we start encoding BB84 in the QUI, let's first do a few simple tasks which will allow us to implement the protocol.

Exercise 2.2 Generate random binary sequences of (10-bit length) using a Hadamard gate followed by a measurement gate. Recall: a H gate places the state into an equal superposition of $|0\rangle$ and $|1\rangle$, and the measurement will randomly collapse it to either $|0\rangle$ or $|1\rangle$. Construct the following ten qubit circuit in QUI:



Press the COMPUTE button (7 times) to produce the following series of 10-bit random sequences and record as per below:

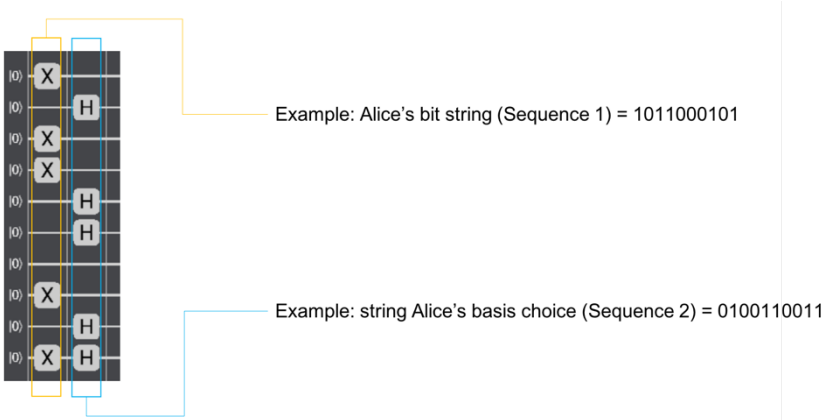
No-eavesdropper case:

Sequence 1:											(Alice's bit string)
Sequence 2:											(Alice's basis choice)
Sequence 3:											(Bob's basis choice)

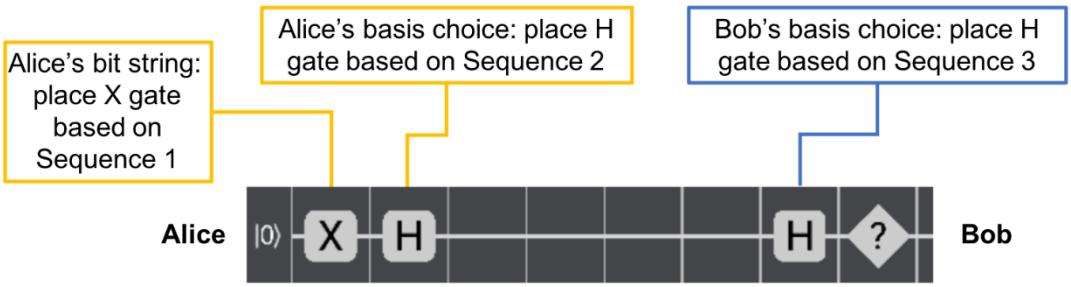
Eavesdropper case:

Sequence 4:											(Alice's bit string)
Sequence 5:											(Alice's basis choice)
Sequence 6:											(Bob's basis choice)
Sequence 7:											(Eve's basis choice)

Exercise 2.3 Implement a 10-qubit circuit in QUI to encode the 10-bit binary "Sequence 1" as per the above, corresponding to Alice's bit string. Now use Sequence 2 corresponding to Alice's random choice of basis encoding: "0" for the Z-basis (no H gate), "1" for the X-basis (H-gate), As an example, a circuit is shown below for Sequence 1=1011000101 and Sequence 2=0100110011:



Exercise 2.4 Code the BB84 protocol for the no-eavesdropper case for all 10 qubits according to your random sequences 1-3 (as per illustration below).



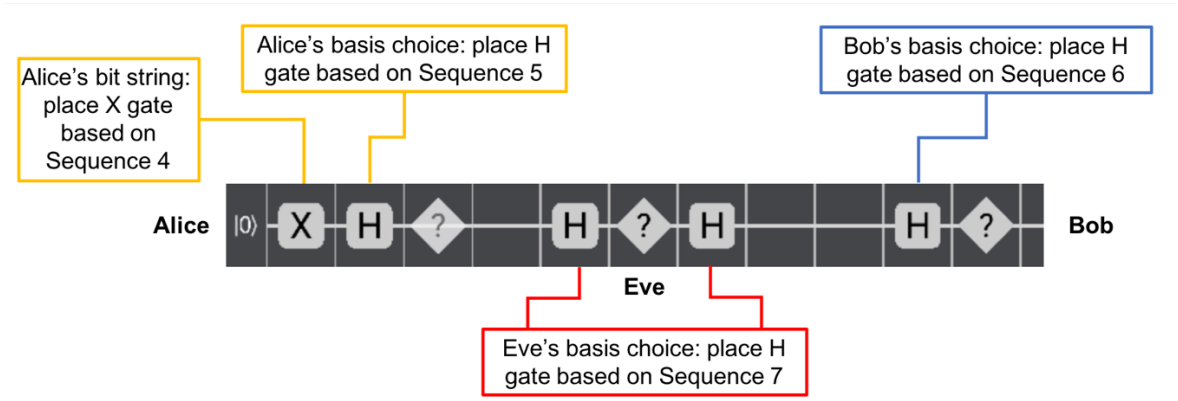
Run the no-eavesdropper-case protocol and fill in the following table:

Table 2.1 – No eavesdropper case

Alice sends (0, 1)	Alice basis (X, Z)	Bob basis (X, Z)	Bob receives (0, 1)	Alice-Bob: correct basis? (yes, no)	Correct bit received? (yes, no)	Kept Bits (0, 1, -)

Number of correct bits received = ____ (= ____ %)

Exercise 2.5 Now add Eve to your circuit, using the sequences 4-7 as indicated below.



Run the eavesdropper-case protocol and fill in the following table:

Table 2.2 – Eavesdropper case

Alice sends (0, 1)	Alice basis (X, Z)	Eve's basis (X, Z)	Eve receives (0, 1)	Bob basis (X, Z)	Bob receives (0, 1)	Alice-Bob: correct basis? (yes, no)	Correct bit received? (yes, no)

Number of correct bits received = ____ (= ____ %)

3 Putting it all together

In the presence of Eve the number of correct bits sent is reduced due to her interference, and if there was no other "noise" in the channel that alone would signal an attack. Without Eve, Alice and Bob would agree in basis choice 50% of the time, and hence 50% of the bits sent can be used for the key. With Eve attacking, the percentage of correct bits will be reduced - the presence of Eve can be detected by comparing the percentage of correct bits for a sub-set of the original string (that will never be used for the key).

Exercise 3.1 Compare the % correct bits received for both the no-Eve and Eve cases.